

PERTEMUAN KE-12: METODE KLASIFIKASI DECISION TREE

A. TUJUAN

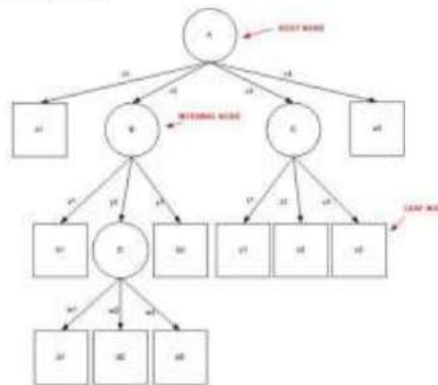
Setelah menempuh praktikum ini diharapkan mahasiswa mampu

1. memahami cara kerja dari metode *Decision Tree*.
2. membuat program metode *Decision Tree* dengan menggunakan Bahasa Pemrograman Python.
3. menggunakan metode *Decision Tree* untuk menyelesaikan masalah klasifikasi.

B. MATERI

Algoritma ID3

Algoritma ID3 merupakan algoritma yang dipergunakan untuk membangun sebuah *decision tree* atau pohon keputusan. Algoritma ini ditemukan oleh J. Ross Quinlan (1979), dengan memanfaatkan Teori Informasi atau Information Theory milik Shanon. ID3 sendiri merupakan singkatan dari Iterative Dichotomiser 3. *Decision tree* menggunakan struktur hierarki untuk pembelajaran supervised. Proses dari *decision tree* dimulai dari root node hingga *leaf node* yang dilakukan secara rekursif. Di mana setiap percabangan menyatakan suatu kondisi yang harus dipenuhi dan pada setiap ujung pohon menyatakan kelas dari suatu data. Proses dalam *decision tree* yaitu mengubah bentuk data (tabel) menjadi model pohon (tree) kemudian mengubah model pohon tersebut menjadi aturan (rule). Dengan pendekatan ini, salah satu kelemahan algoritma dari *decision tree*, adalah faktor skalabilitas dimana algoritma tersebut hanya dapat digunakan untuk menangani sampel-sampel yang dapat disimpan secara keseluruhan dan pada waktu yang bersamaan di memori. Algoritma ID3 tidak pernah melakukan backtracking untuk merevisi keputusan pemilihan attribute yang telah dilakukan sebelumnya. ID3 hanya menangani nilai-nilai attribute yang sedikit dan diskret, tetapi algoritma modifikasinya, algoritma C4.5 (1993), selanjutnya mampu menangani nilai attribute kontinu.



Gambar 12.1. Arsitektur Pohon Keputusan

Arsitektur pohon keputusan dibuat menyerupai bentuk pohon, dimana pada umumnya sebuah pohon terdapat akar (root), cabang dan daun (leaf). Pada pohon keputusan juga terdiri dari tiga bagian sebagai berikut :

- a. **Root node** atau node akar merupakan node yang terletak paling atas dari suatu pohon.
- b. **Internal Node** ini merupakan node percabangan, dimana pada node ini hanya terdapat satu input dan mempunyai minimal dua output.
- c. **Leaf Node** ini merupakan node akhir, hanya memiliki satu input, dan tidak memiliki output. Pada pohon keputusan setiap leaf node menandai label kelas.

Pada pohon keputusan di setiap percabangan menyatakan kondisi yang harus dipenuhi dan tiap ujung pohon menyatakan nilai kelas data. Gambar 12.1 merupakan bentuk arsitektur pohon. Adapun langkah-langkah dalam konstruksi pohon keputusan adalah sebagai berikut :

1. Pohon dimulai dengan sebuah simpul yang merepresentasikan sampel data pelatihan yaitu dengan membuat simpul akar.

2. Jika semua sampel berada dalam kelas yang sama, maka simpul ini menjadi daun dan dilabeli menjadi kelas. Jika tidak, information gain akan digunakan untuk memilih atribut terbaik dalam memisahkan data sampel menjadi kelas-kelas individu.
3. Cabang akan dibuat untuk setiap nilai pada atribut dan data sampel akan dipartisi lagi.
4. Algoritma ini menggunakan proses rekursif untuk membentuk pohon keputusan pada setiap data partisi. Jika sebuah atribut sudah digunakan disebuah simpul, maka atribut ini tidak akan digunakan lagi di simpul anak-anaknya.
5. Proses ini berhenti jika dicapai kondisi seperti berikut :
 - Semua sampel pada simpul berada di dalam satu kelas
 - Tidak ada atribut lainnya yang dapat digunakan untuk mempartisi sampel lebih lanjut. Dalam hal ini akan diterapkan suara terbanyak. Ini berarti mengubah sebuah simpul menjadi daun dan melabelinya dengan kelas pada suara terbanyak.

Algoritma pada metode ini menggunakan konsep dari entropi. Konsep Entropi yang digunakan untuk mengukur “seberapa informatifnya” sebuah node (yang biasanya disebut seberapa baiknya). $Entropi(S) = 0$, jika semua contoh pada S berada dalam kelas yang sama. $Entropi(S) = 1$, jika jumlah contoh positif dan jumlah contoh negatif dalam S adalah sama. $0 < Entropi(S) < 1$, jika jumlah contoh positif dan negatif dalam S tidak sama. Entropi didefinisikan oleh persamaan

$$Entropi(S) = \sum_{j=1}^k -p_j \log_2 p_j$$

dimana, S adalah himpunan (dataset) kasus, k adalah banyaknya partisi S , p_j adalah probabilitas yang di dapat dari Jumlah(Y_i) dibagi total kasus. Setelah mendapat nilai entropi, pemilihan atribut dilakukan dengan nilai information gain terbesar. Information gain didefinisikan oleh

$$Gain(A) = Entropi(S) - \sum_{j=1}^k \frac{|S_j|}{|S|} Entropi(S_j)$$

dimana, S = ruang (data) sample yang digunakan untuk training, A = atribut, $|S_i|$ = jumlah sample untuk nilai V , $|S|$ = jumlah seluruh sample data, $Entropi(S_i)$ = entropy untuk sample-sample yang memiliki nilai i

Contoh 12.1

Data yang telah ada pada Tabel dibawah akan digunakan untuk membentuk pohon keputusan dimana memiliki atribut-atribut seperti **Cuaca**, **Suhu**, **Kelembaban**, dan **Berangin**. Setiap atribut memiliki nilai. Sedangkan kelasnya ada pada kolom Main yaitu kelas “**Tidak**” dan kelas “**Ya**”. Kemudian data tersebut dianalisis; dataset tersebut memiliki 14 kasus yang terdiri 10 “Ya” dan 4 “Tidak” pada kolom Main.

Tabel 12.1 Learning dataset

No	Cuaca	Suhu	Kelembaban	Berangin	Main
1	Cerah	Panas	Tinggi	Salah	Tidak
2	Cerah	Panas	Tinggi	Benar	Tidak
3	Berawan	Panas	Tinggi	Salah	Ya
4	Hujan	Sejuk	Tinggi	Salah	Ya
5	Hujan	Dingin	Normal	Salah	Ya
6	Hujan	Dingin	Normal	Benar	Ya
7	Berawan	Dingin	Normal	Benar	Ya
8	Cerah	Sejuk	Tinggi	Salah	Tidak
9	Cerah	Dingin	Normal	Salah	Ya
10	Hujan	Sejuk	Normal	Salah	Ya
11	Cerah	Sejuk	Normal	Benar	Ya
12	Berawan	Sejuk	Tinggi	Benar	Ya
13	Berawan	Panas	Normal	Salah	Ya
14	Hujan	Sejuk	Tinggi	Benar	Tidak

Kemudian hitung entropi dengan rumus seperti diatas.

$$\text{Entropi (S)} = -(10/14) \times \log_2 (10/14) + -(4/10) \times \log_2 (4/10) = \mathbf{0.863120569}$$

Tabel 12.2. Hasil perhitungan entropi

Total Kasus	Jumlah Ya	Jumlah Tidak	Entropi Total
14	10	4	0.863120569

Setelah mendapatkan entropi dari keseluruhan kasus, lakukan analisis pada setiap atribut dan nilai-nilainya dan hitung entropinya.

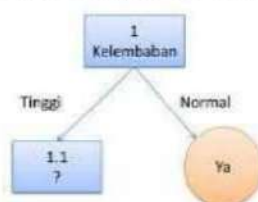
Tabel 12.3. Analisis atribut, nilai, banyaknya kejadian, entropi dan gain

Node	Atribut	Nilai	Sum (Nilai)	Sum(Ya)	Sum(Tidak)	Entropi	Gain
1	Cuaca	Berawan	4	4	0	0	
		Hujan	5	4	1	0,72198	
		Cerah	5	2	3	0,97095	
	Suhu	Dingin	4	4	0	0	
		Panas	4	2	2	1	
		Sejuk	6	4	2	0,91829	
	Kelembaban	Tinggi	7	3	4	0,9852	
		Normal	7	7	0	0	
	Berangin	Salah	8	6	2	0,8112	
		Benar	6	2	4	0,9182	
						0,005977	

Setelah mendapatkan nilai entropy, berikutnya hitung nilai information gain dari setiap variabel.

$$\text{Gain (Cuaca)} = 0.863120569 - ((4/10) \times 0 + (5/14) \times 0.721928095 + (5/14) \times 0.970950594) = \mathbf{0.258521037}$$

Hitung pula Gain (Suhu), Gain (Kelembaban), dan Gain (Berangin). **Karena nilai gain terbesar adalah Gain (Kelembaban), maka atribut “Kelembaban” menjadi node akar (root node).** Kemudian pada “Kelembaban” normal, memiliki 7 kasus dan semuanya memiliki jawaban Ya (Sum(Total) / Sum(Ya) = 7/7 = 1). Dengan demikian “Kelembaban” normal menjadi daun atau *leaf*.



Gambar 12.2. pohon keputusan node 1 (root node)

Berdasarkan pembentukan pohon keputusan node 1 (root node), Node 1.1 akan dianalisis lebih lanjut. Untuk mempermudah, Tabel dibawah difilter, dengan mengambil data yang memiliki “Kelembaban” = Tinggi.

Tabel 12.4. Atribut data kelembaban = tinggi

Cuaca	Suhu	Kelembaban	Berangin	Main
Cerah	Panas	Tinggi	Salah	Tidak
Cerah	Panas	Tinggi	Benar	Tidak
Berawan	Panas	Tinggi	Salah	Ya
Hujan	Sejuk	Tinggi	Salah	Ya
Cerah	Sejuk	Tinggi	Salah	Tidak

Berawan	Sejuk	Tinggi	Benar	Ya
Hujan	Sejuk	Tinggi	Benar	Tidak

Kemudian dihitung nilai entropi atribut “Kelembaban” Tinggi dan entropi setiap atribut serta gainnya. Setelah itu tentukan pilih atribut yang memiliki gain tertinggi untuk dibuatkan node berikutnya.

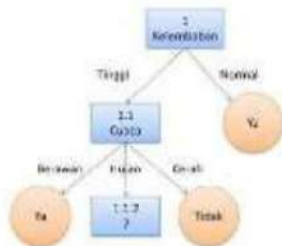
Tabel 12.5. Hasil perhitungan entropi

Kelembaban Tinggi	Jumlah Ya	Jumlah Tidak	Entropi Total
7	3	4	0,9852

Tabel 12.6. Hasil analisis node 1.1

Node	Atribut	Nilai	Sum (Nilai)	Sum(Ya)	Sum(Tidak)	Entropi	Gain
1	Cuaca	Berawan	2	2	0	0	0,699513
		Hujan	2	1	1	1	
		Cerah	3	0	3	0	
	Suhu	Dingin	0	0	0	0	0,02024
		Panas	3	1	2	0,91829	
		Sejuk	4	2	2	1	
	Berangin	Salah	4	2	2	1	0,020244
		Benar	3	2	1	0,9182	
						0,020244	

Gain tertinggi yang didapat ada pada atribut “Cuaca”, dan Nilai yang dijadikan daun atau *leaf* adalah Berawan dan Cerah. Jika divualisasi maka pohon keputusan tampak seperti Gambar 12.3. Untuk menganalisis node 1.1.2, lakukan lagi langkah-langkah yang sama seperti sebelumnya hingga semua node membentuk node *leaf*.



Gambar 12.3 Pohon keputusan analisis node 1.1

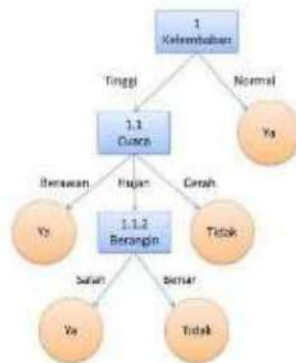
Tabel 12.7 Hasil analisis node 1.1.2.

No	Cuaca	Suhu	Kelembaban	Berangin	Main
1	Hujan	Sejuk	Tinggi	Salah	Ya
2	Hujan	Sejuk	Tinggi	Benar	Tidak

Kelembaban Tinggi & Hujan	jumlah ya	Jumlah Tidak	Entropi
2	1	1	1

Node	Atribut	Nilai	Sum (Nilai)	Sum(Ya)	Sum(Tidak)	Entropi	Gain
	Suhu	Dingin	0	0	0	0	

		Panas	0	0	0	0	
		Sejuk	2	1	1	1	
	Berangin	Salah	1	1	0	0	0
		Benar	1	0	1	0	
							1



Gambar 12.4. Pohon keputusan akhir

Algoritma C4.5

Algoritma data mining C4.5 merupakan salah satu algoritma yang digunakan untuk melakukan klasifikasi atau segmentasi atau pengelompokan dan bersifat prediktif. Klasifikasi merupakan salah satu proses pada data mining yang bertujuan untuk menemukan pola yang berharga dari data yang berukuran relatif besar hingga sangat besar. Algoritma C4.5 sendiri merupakan pengembangan dari algoritma ID3.

Algoritma C4.5 mempunyai prinsip dasar kerja yang sama dengan algoritma ID3. Beberapa perbedaannya antara lain :

a. **Mampu Menangani Atribut dengan Tipe Diskrit atau Kontinu**

Pemilihan atribut pada algoritma induksi *decision tree* menggunakan ukuran berdasarkan entropy yang dikenal dengan information gain sebagai sebuah heuristic untuk memilih atribut yang merupakan bagian terbaik dari contoh ke dalam kelas. Semua atribut adalah bersifat kategori yang bernilai diskrit. Atribut dengan nilai *continuous* harus didiskritkan.

Tabel 12.8. Atribut kontinu

Outlook	Temperatur	Humidity	Windy	Play
Sunny	85	85	False	No
Sunny	80	90	True	No
Overcast	83	86	False	Yes
Rainy	75	80	False	Yes
....	...			

Diskritisasi atribut bertujuan untuk mempermudah pengelompokan nilai berdasarkan kriteria yang telah ditetapkan. Hal ini juga bertujuan untuk menyederhanakan permasalahan dan meningkatkan akurasi dalam proses pembelajaran. Terdapat beberapa teknik yang bisa dilakukan, diantaranya 'binning', yang mendefinisikan kumpulan kelas nominal untuk setiap atribut, kemudian menetapkan setiap nilai atribut ke dalam salah satu kelas. Misalnya, jika domain atribut numerik mempunyai nilai dari 0 sampai dengan 100, domain tersebut dapat dibagi menjadi empat bin (0..24, 25..49, 50..74, 75..100). Setiap nilai atribut akan dikonversi menjadi atribut nominal yang berkorespondensi dengan salah satu bin. Oleh karena itu metode binning disebut *unsupervised discretization method*. Teknik berikutnya dapat dilakukan dengan

pengukuran entropi, dengan mempertimbangkan semua kemungkinan pembagian dan cari pemotongan terbaik.



Gambar 12.5. Split kontinu

Langkah pertama adalah urutkan nilai atribut kontinu beserta nilai kelas utama nya. Kemudian lakukan pemotongan dengan pertimbangan pembagian jumlah kelas utama 50:50 atau mendekati, untuk memerkirakan mendapat nilai entropi tertinggi atau mendekati 1.

Contoh:

$temperature < 71.5$: $yes/4$, $no/2$

$temperature \geq 71.5$: $yes/5$, $no/3$

$Info([4,2],[5,3]) = 6/14 \text{ info}([4,2]) + 8/14 \text{ info}([5,3]) = 0.939$

Teknik lain, normalisasi, juga dapat dilakukan dengan cara mencari nilai rata dan standart deviasi pada rentang atribut kontinu.

b. Mampu Menangani Atribut yang Kosong (Missing Value)

Nilai kosong pada sebuah dataset wajib kita isi terlebih dahulu sebelum diproses untuk tahap *machine learning* atau bentuk kedalam sebuah model *decision tree*.

Tabel 12.9. Missing value C4.5-1

BI-RAD	Age	Shape	Margin	Density	Class
4	48	4	5	?	1
5	67	3	5	3	1
5	57	4	4	3	1
5	60	?	5	1	1
4	53	?	4	3	1
4	28	1	1	3	0
4	70	?	2	3	0
2	66	1	1	?	0
5	63	3	?	3	0
4	78	1	1	1	0

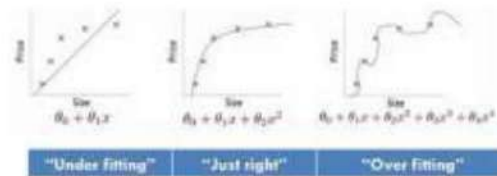
Cara paling mudah dalam pengisian atribut kosong adalah dengan memberikan nilai berdasar nilai yang paling banyak atau dominan dalam atribut tersebut. Contoh, atribut Shape baris ketiga dan keempat kita beri nilai 4 dikarenakan pada kelompok class 1 mayoritas atribut Shape memiliki angka 4. Sedangkan pada class 0, baris 7 kita berikan nilai 1.

Tabel 12.10. Missing-value-c4.5-2

BI-RAD	Age	Shape	Margin	Density	Class
4	48	4	5	3	1
5	67	3	5	3	1
5	57	4	4	3	1
5	60	4	5	1	1
4	53	4	4	3	1
4	28	1	1	3	0
4	70	1	2	3	0
2	66	1	1	3	0
5	63	3	?	3	0
4	78	1	1	1	0

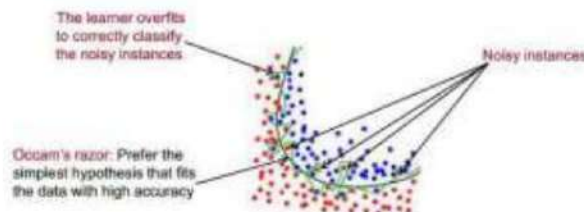
c. Pemangkasan Pohon Keputusan

Pada saat pembangunan pohon keputusan, banyaknya cabang mungkin mencerminkan adanya *noise* atau *outlier* pada training data. Pemangkasan pohon dapat dilakukan untuk mengenali dan menghapus cabang-cabang tersebut. Pohon yang dipangkas akan menjadi lebih kecil dan lebih mudah dipahami. Pohon semacam itu biasanya juga menjadi lebih cepat dan lebih baik dalam melakukan klasifikasi.



Gambar 12.6 overfitting vs underfitting

Jika suatu fungsi atau dataset memiliki terlalu banyak atribut (fitur), model pelatihan yang terbentuk akan menghasilkan akurasi yang baik, namun akan sangat mungkin mengalami kegagalan untuk memprediksi data masukan baru seperti pada contoh diatas. Pemangkasan pohon juga dapat digunakan untuk mengatasi overfitting. Overfitting terjadi karena ada noise data training, yaitu data yang tidak relevan sehingga mengakibatkan pohon memiliki subtree yang panjang dan tidak seimbang. Misal internal node memiliki kelas YA = 5 dan TIDAK = 1. Data yang berada pada kelas TIDAK merupakan noise, sehingga apabila data tersebut diolah akan menghasilkan pohon dengan subtree yang panjang. Overfitting juga dapat terjadi karena data training yang sedikit.



Gambar 12.7. Overfitting

Ada dua metode dalam melakukan pemangkasan dalam pohon keputusan, yaitu:

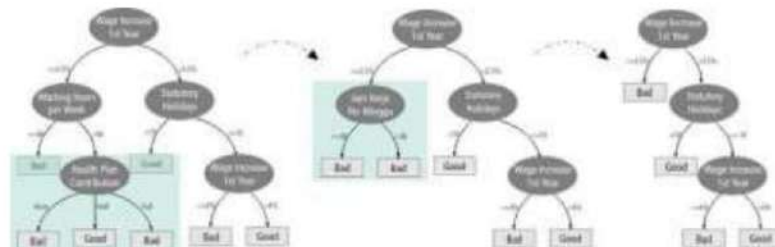
- Prepruning**, yaitu menghentikan pembangunan suatu subtree lebih awal, yaitu dengan memutuskan untuk tidak lebih jauh mempartisi data training. Pada pendekatan prepruning, sebuah pohon dipangkas dengan cara menghentikan pembangunannya jika partisi yang akan dibuat dianggap tidak signifikan. Untuk mengetahui nilai parameter apakah akan dilakukan expanding atau pruning dapat menggunakan metode *chi-squared*.
- Postpruning**, yaitu menyederhanakan pohon dengan cara membuang beberapa cabang subtree setelah pohon selesai dibangun. Metode postpruning ini merupakan metode standard untuk algoritma C4.5.

Reduced Error Pruning merupakan salah satu algoritma postpruning. Algoritma ini membagi data menjadi dua, yaitu training data dan test data. Training data adalah data yang digunakan untuk membentuk pohon keputusan, sedangkan test data digunakan untuk menghitung nilai error rate pada pohon setelah dipangkas.

Cara kerja REP adalah dengan memangkas internal node yang dimulai dari internal node paling bawah ke atas. Pemangkasan dilakukan dengan cara mengganti atribut dengan leaf node yang memiliki kelas yang dominan muncul. Setelah itu test data diproses menggunakan rule hasil pemangkasan, kemudian dihitung nilai error ratenya. Test data juga diproses dengan rule awal, yaitu rule yang terbentuk sebelum pohon dipangkas, kemudian dihitung nilai error ratenya. Apabila nilai error rate yang dihasilkan dari pemangkasan pohon lebih kecil, maka pemangkasan dilakukan. Reduced error pruning diformulasikan oleh

$$e = \frac{r + \frac{z^2}{2n}}{1 + \frac{z^2}{n}} + z \sqrt{\frac{r}{n} - \frac{r^2}{n} + \frac{z^2}{4n^2}}$$

dimana, r = nilai perbandingan error rate, n = total sample dan $z = \Phi^{-1}(c)$.



Gambar 12.8 Post pruning tree

Pada contoh diatas dapat diketahui bahwa atribut “Health Plan Contribution” banyak memiliki class Bad, maka dari itu dipangkas menjadi leaf node Bad untuk keseluruhan atribut tersebut, dan seterusnya.

d. **Pemilihan Atribut yang Dilakukan Menggunakan Gain Ratio**

Quinlan memperkenalkan metode C4.5, dimana untuk pemilihan split atribut menggunakan metode Gain Ratio (GR) menggantikan Information Gain (IG).

Information gain pada ID3 lebih mengutamakan pengujian yang menghasilkan banyak keluaran. Dengan kata lain, atribut yang memiliki banyak nilailah yang dipilih sebagai splitting atribut. Sebagai contoh, pembagian terhadap atribut yang berfungsi sebagai unique identifier, seperti product_ID, akan menghasilkan keluaran dalam jumlah yang banyak, di mana setiap keluaran hanya terdiri dari satu tuple.

Tabel 12.11. Data set

No	Cuaca	Suhu	Kelembaban	Berangin	Main
1	Cerah	Panas	Tinggi	Salah	Tidak
2	Cerah	Panas	Tinggi	Benar	Tidak
3	Berawan	Panas	Tinggi	Salah	Ya
4	Hujan	Sejuk	Tinggi	Salah	Ya
5	Hujan	Dingin	Normal	Salah	Ya
6	Hujan	Dingin	Normal	Benar	Ya
7	Berawan	Dingin	Normal	Benar	Ya
8	Cerah	Sejuk	Tinggi	Salah	Tidak
9	Cerah	Dingin	Normal	Salah	Ya
10	Hujan	Sejuk	Normal	Salah	Ya
11	Cerah	Sejuk	Normal	Benar	Ya
12	Berawan	Sejuk	Tinggi	Benar	Ya
13	Berawan	Panas	Normal	Salah	Ya
14	Hujan	Sejuk	Tinggi	Benar	Tidak

Partisi semacam ini tentu saja bersifat pure, sehingga informasi yang dibutuhkan untuk mengklasifikasi D berdasarkan partisi seperti ini adalah sebesar $\text{Info}_{\text{product_ID}}(D) = 0$. Sebagai akibatnya, information gain yang dimiliki atribut product_ID menjadi maksimal. Padahal, jelas sekali terlihat bahwa partisi semacam ini tidaklah berguna. Karena itu algoritma C4.5 yang merupakan suksesor dari ID3 menggunakan gain ratio untuk memperbaiki information gain, dengan rumus gain ratio (Larose, 2009).

$\text{info}([0,1]) + \text{info}([0,1]) + \text{info}([0,1]) + \dots + \text{info}([0,1])$ nilai information gain akan mencapai maksimal untuk attribute ID code.

3. Entropy, Information Gain & Gain Ratio

Pemilihan atribut yang baik adalah atribut yang memungkinkan untuk mendapatkan decision tree yang paling kecil ukurannya. Atau atribut yang bisa memisahkan obyek menurut kelasnya. Secara heuristik

atribut yang dipilih adalah atribut yang menghasilkan simpul yang paling "purest" (paling bersih). Ukuran purity dinyatakan dengan tingkat impurity, dan untuk menghitungnya, dapat dilakukan dengan menggunakan konsep Entropy, Entropy menyatakan impurity suatu kumpulan

Formula mencari entropi sebagai berikut :

$$Entropi(S) = \sum_{j=1}^k -p_j \log_2 p_j$$

dimana S adalah himpunan (dataset) kasus, k adalah banyaknya partisi S , dan p_j adalah probabilitas yang di dapat dari jumlah Ya dibagi Total Kasus.

Information gain adalah kriteria yang paling populer untuk pemilihan atribut. Algoritma C4.5 adalah pengembangan dari algoritma ID3. Oleh karena pengembangan tersebut algoritma C4.5 mempunyai prinsip dasar kerja yang sama dengan algoritma ID3. Hanya saja dalam algoritma C4.5 pemilihan atribut dilakukan dengan menggunakan Gain Ratio dengan rumus :

$$gain\ ratio(a) = \frac{gain(a)}{split(a)}$$

dimana a = atribut, $gain(a)$ = information gain pada atribut a , $Split(a)$ = split information pada atribut a Atribut dengan nilai Gain Ratio tertinggi dipilih sebagai atribut test untuk simpul. Dengan gain adalah information gain. Pendekatan ini menerapkan normalisasi pada information gain dengan menggunakan apa yang disebut sebagai split information. SplitInfo menyatakan entropy atau informasi potensial dengan rumus :

$$splitInfo(S, A) = -\sum_{i=1}^n \frac{S_i}{S} \log_2 \frac{S_i}{S}$$

dimana S = ruang (data) sample yang digunakan untuk training, A = atribut, S_i = jumlah sample untuk atribut i .

Information gain diformulasikan oleh

$$Gain(A) = Entropi(S) - \sum_{j=1}^k \frac{|S_j|}{|S|} Entropi(S_j)$$

dimana S = ruang (data) sample yang digunakan untuk training, A = atribut, $|S_i|$ = jumlah sample untuk nilai V , $|S|$ = jumlah seluruh sample data dan $Entropi(S_i)$ = entropy untuk sample-sample yang memiliki nilai i

Alasan penggunaan $gain\ ratio(a)$ pada C4.5 (bukan $gain(a)$) sebagai kriteria pada pemilihan atribut adalah gain ternyata bias terhadap atribut yagn memiliki banyak nilai unik.

Contoh 11.3

Contoh Kasus: Klasifikasi Iris Dataset

Iris dataset adalah dataset yang berisi informasi tentang panjang dan lebar sepal dan petal dari tiga jenis bunga iris (Setosa, Versicolor, Virginica). Tujuan dari model klasifikasi ini adalah untuk memprediksi jenis bunga iris berdasarkan panjang dan lebar sepal dan petal.

Pada implementasi **Decision Tree** menggunakan **Iris Dataset**, langkah pertama yang harus dilakukan adalah **praproses data**, di mana kita memisahkan fitur (X) dan label (y). Fitur dalam dataset ini meliputi panjang sepal (sepal length), lebar sepal (sepal width), panjang petal (petal length), dan lebar petal (petal width), sementara labelnya terdiri dari tiga kelas bunga iris, yaitu Setosa, Versicolor, dan Virginica. Setelah memisahkan fitur dan label, data kemudian dibagi menjadi dua bagian: data pelatihan (training) untuk membangun model dan data uji (test) untuk mengevaluasi performa model.

Selanjutnya, langkah kedua adalah **membangun model Decision Tree**, yang dapat menggunakan algoritma **ID3** atau **CART**. Dalam hal ini, metrik pembelahan yang digunakan bisa berupa **information gain** (untuk ID3) atau **Gini Impurity** (untuk CART). Kedua metrik ini berfungsi untuk memilih atribut terbaik yang akan membagi dataset pada setiap simpul pohon berdasarkan seberapa besar pengaruhnya terhadap pembagian data.

Setelah model terbentuk, tahap berikutnya adalah **evaluasi model**. Evaluasi dilakukan dengan mengukur kinerja model menggunakan metrik **accuracy**, **precision**, **recall**, dan **F1-score**. Accuracy mengukur seberapa sering model benar dalam melakukan prediksi, sementara precision dan recall memberikan gambaran lebih mendalam tentang bagaimana model mengidentifikasi masing-masing kelas. F1-score, yang merupakan rata-rata harmonis dari precision dan recall, digunakan untuk memberikan ukuran keseluruhan performa model, terutama saat kelas data tidak seimbang.

Jawaban

Kode Program 12.12. Program Decision Tree

No	Kode Program
1	<code>import pandas as pd</code>
2	<code>from sklearn.datasets import load_iris</code>
3	<code>from sklearn.model_selection import train_test_split</code>
4	<code>from sklearn.tree import DecisionTreeClassifier</code>
5	<code>from sklearn.metrics import accuracy_score, classification_report,</code>
6	<code>confusion_matrix</code>
7	<code>from sklearn.tree import plot_tree</code>
8	<code>import matplotlib.pyplot as plt</code>
9	<code># Load Iris Dataset</code>
10	<code>iris = load_iris()</code>
11	<code>X = iris.data</code>
12	<code>y = iris.target</code>
13	<code># Split Data menjadi training dan test</code>
14	<code>X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,</code>
15	<code>random_state=42)</code>
16	<code># Bangun Model Decision Tree</code>
17	<code>dtree = DecisionTreeClassifier(criterion='gini', random_state=42)</code>
18	<code>dtree.fit(X_train, y_train)</code>
19	<code># Prediksi dengan data uji</code>
20	<code>y_pred = dtree.predict(X_test)</code>
21	<code># Evaluasi Model</code>
22	<code>print("Accuracy:", accuracy_score(y_test, y_pred))</code>
23	<code>print("Classification Report:")</code>
24	<code>print(classification_report(y_test, y_pred))</code>
25	<code>print("Confusion Matrix:")</code>
26	<code>print(confusion_matrix(y_test, y_pred))</code>
27	<code># Visualisasi Pohon Keputusan</code>
28	<code>plt.figure(figsize=(12,8))</code>
29	<code>plot_tree(dtree, filled=True, feature_names=iris.feature_names,</code>
40	<code>class_names=iris.target_names)</code>
41	<code>plt.title("Decision Tree - Iris Dataset")</code>
42	<code>plt.show()</code>

```

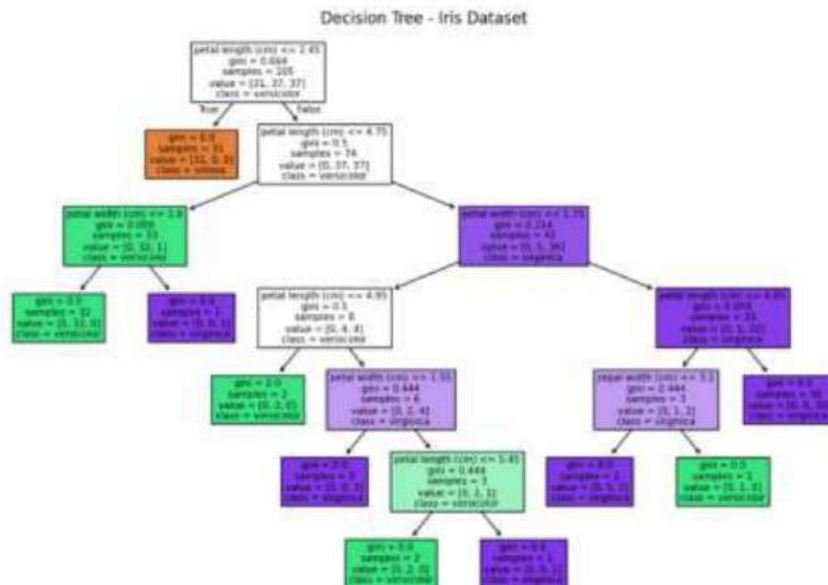
Accuracy: 1.0
Classification Report:
              precision    recall  f1-score   support

     0         1.00      1.00      1.00        19
     1         1.00      1.00      1.00        13
     2         1.00      1.00      1.00        13

 accuracy          1.00
 macro avg          1.00
weighted avg          1.00

Confusion Matrix:
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]

```

Berdasarkan hasil evaluasi model **Decision Tree** yang diterapkan pada **Iris Dataset**, kita mendapatkan **accuracy** sebesar 1.0, yang berarti model berhasil melakukan prediksi dengan benar pada semua data uji. **Classification Report** menunjukkan bahwa precision, recall, dan F1-score untuk setiap kelas (Setosa, Versicolor, dan Virginica) semuanya bernilai 1.0, yang menandakan bahwa model sangat akurat dalam mengklasifikasikan setiap kelas tanpa kesalahan. **Support** menunjukkan jumlah sampel untuk setiap kelas, dengan 19 sampel untuk Setosa, 13 untuk Versicolor, dan 13 untuk Virginica, yang semuanya berhasil diprediksi dengan sempurna. **Confusion Matrix** mengonfirmasi hasil ini, dengan nilai diagonal yang menunjukkan jumlah prediksi yang benar untuk setiap kelas, dan semua nilai non-diagonal adalah 0, yang menunjukkan bahwa tidak ada kesalahan dalam klasifikasi. Matriks ini memperlihatkan bahwa model sangat efektif dalam memisahkan ketiga jenis bunga iris. Dari **visualisasi pohon keputusan**, kita bisa melihat bagaimana model memecah dataset berdasarkan fitur-fitur seperti panjang dan lebar sepal dan petal. Setiap simpul pohon mewakili keputusan untuk membagi data berdasarkan nilai ambang tertentu dari fitur yang dipilih, dan cabang-cabangnya mengarah ke keputusan akhir (daun) yang menunjukkan kelas bunga iris yang diprediksi. Visualisasi ini membantu memahami bagaimana pohon keputusan membuat prediksi, serta fitur mana yang paling signifikan dalam membedakan antara kelas-kelas tersebut. Jika pohon terlalu kompleks atau terlalu banyak cabang, bisa jadi indikasi bahwa model mengalami **overfitting**, meskipun pada kasus ini dengan **accuracy** yang sempurna, model tampaknya berhasil menggeneralisasi dengan baik pada data uji.

C. TUGAS/LATIHAN SOAL

Titanic Dataset: <https://www.kaggle.com/c/titanic>

1. **Praproses Data:**
 - Pisahkan fitur dan label.
 - Lakukan pembagian data menjadi data pelatihan dan data uji.
 - Pastikan dataset tidak memiliki missing values.
2. **Implementasi Decision Tree:**
 - Bangun model Decision Tree menggunakan **criterion='gini'** atau **criterion='entropy'**.
 - Tentukan kedalaman pohon dan jumlah minimum sampel pada daun untuk menghindari overfitting.
3. **Evaluasi Model:**
 - Hitung **accuracy**, **precision**, **recall**, dan **F1-score** untuk mengevaluasi kinerja model.
4. **Pruning Pohon Keputusan:**
 - Terapkan **pre-pruning** atau **post-pruning** untuk meningkatkan hasil model.
5. **Visualisasi Pohon Keputusan:**
 - Gunakan **plot_tree()** dari **sklearn.tree** untuk memvisualisasikan pohon keputusan.